Clean code : Rules to follow, standards standards, According to the programming language or the programmer, snake_case, Camel case

Programing Paradigm : Structure Programming (Procedural), OOP)

VCS, Git we manage versioning of software development artifacts code, test case, data files,

OOP Concepts (Pillars of OOP) Encapsulation, Abstraction, Inheritance, Polymorphism
Object and Classes

- Encapsulation : classes, packages, libraries, it is implemented using private
- Abstraction : abstract methods, interfaces, think in what really matters for my application domain, it is implemented using public, abstract, interface
- Inheritance, take advantage of what other classes have already done
- Polymorphism : use the same name for different, but related activities

Constructors : used to initialize / instantiate objects of a class

Person person = new Person(1, "Leslye Quichan, 4076);

int a;
not initialized

Objects and classes : UML Class Diagram
Functionality and user : Use Case Diagram
Classes : attributes and methods

Classes relationships in class Diagrams

- Dependencies : The function of one class uses object of another class, either as parameters or as return data type, or even inside the methods
- Associations : one object of a class contains or has and attribute / object of another class.
- Compositions : the life line of the composed objects depends on the life of the composed objects.

VERZATIL

Escaneado con CamScanner

- Aggregations: every object depends on itself to exist

Exepcions, handle users' errors

setters → setting values

getters → obtaining values of attribute

primitive data types: int, float

Data files → data persistencies

COLLECTIONS

Programming interfaces

Set of rules and protocols that allow computer programs to talk to each other contact/blue print that defines a set of methods that we must implement so they can be used by other objects

A set of fuctions prototypes (function declarations)

```
Float (float addend, float addend2); // declaration / prototype
public abstract class Operation {

  float operand1;
  float operand 2;
  float result;

  public float add (adden 1, float addent 2);
  public float substract (float minuhend, float subtrahed);
  public float multiply (float multiplicand, float multiplier);
  public float divide (float dvident, float divisor);
```

3 Not use second unit.

```
Public interface Operation {
  interface → not attribut only methods
  pub
```

```
Student students[];
Students = new Student[25]; // static size
```

Escaneado con CamScanner